# ROME Assignment - Coverage Integration

## 1. Model and settings

Model: **nvidia/nemotron-3-ultra-550b-a55b**, called through NVIDIA's OpenAI-compatible API. I used the original ROME `ChatGPT` class and pointed it at NVIDIA with `OPENAI_BASE_URL`. Temperature and top-p are the API defaults. I added retries for API errors. ROME settings: max 10 iterations per module, previous modules given to the LLM as interfaces only, compiler warnings count as failures. PPA optimizer: 5 iterations per mux, Yosys with no liberty file. The same model is used for RTL generation and for testbench refinement.

## 2. Hierarchy and interfaces

| Module | Interface | Built from |
|---|---|---|
| half_adder | input a, b; output sum, cout | sum = a ^ b, cout = a & b |
| full_adder | input a, b, cin; output sum, cout | 2 x half_adder + OR for carry |
| adder4 | input [3:0] a, b; input cin; output [3:0] sum; output cout | 4 x full_adder, ripple carry |
| adder8 (top) | input [7:0] a, b; input cin; output [7:0] sum; output cout | 2 x adder4, low carry feeds high |

All ports are `wire`. Each module has a self-checking baseline TB. Expected values come from Verilog arithmetic ({cout,sum} = a + b + cin), not hard-coded numbers. Each TB prints `All tests passed!` only if every check passes. The half and full adder TBs are exhaustive. adder4 has 8 tests. The adder8 baseline has 8 small tests on purpose, with no carry out of bit 7.

## 3. Final prompts

Top module, adder8 (the prompt ROME sent; it passed on iteration 0):

```
// We are generating a 8-bit ripple-carry adder built from two adder4 instances (carry out of the low nibble feeds the high nibble) hierarchica
// Use Verilog-2001 only.

// Previously generated modules are shown below for reference only.
// You may instantiate them, but you must NOT repeat their definitions.

// File: ./half_adder/half_adder.v
module half_adder(input wire a, input wire b, output wire sum, output wire cout);
endmodule

// File: ./full_adder/full_adder.v
module full_adder(
  input  wire a,
  input  wire b,
  input  wire cin,
  output wire sum,
  output wire cout
);
endmodule

// File: ./adder4/adder4.v
module adder4 (
  input  wire [3:0] a,
  input  wire [3:0] b,
  input  wire       cin,
  output wire [3:0] sum,
  output wire       cout
);
endmodule

// IMPORTANT: Output ONLY the new module requested below.
// Do NOT repeat any previous modules.
// Do NOT generate explanations, Markdown fences, or a testbench.
// Now generate: 8-bit ripple-carry adder built from two adder4 instances (carry out of the low nibble feeds the high nibble)

module adder8(input wire [7:0] a, input wire [7:0] b, input wire cin, output wire [7:0] sum, output wire cout);
// Insert code here
endmodule
```

Submodule, full_adder (also passed on iteration 0):

```
// We are generating a 8-bit ripple-carry adder built from two adder4 instances (carry out of the low nibble feeds the high nibble) hierarchica
// Use Verilog-2001 only.

// Previously generated modules are shown below for reference only.
// You may instantiate them, but you must NOT repeat their definitions.

// File: ./half_adder/half_adder.v
module half_adder(input wire a, input wire b, output wire sum, output wire cout);
endmodule

// IMPORTANT: Output ONLY the new module requested below.
// Do NOT repeat any previous modules.
// Do NOT generate explanations, Markdown fences, or a testbench.
// Now generate: 1-bit full adder built from two half_adder instances and an OR gate for the carry out
```

```
module full_adder(input wire a, input wire b, input wire cin, output wire sum, output wire cout);
// Insert code here
endmodule
```

## 4. RTL generation and repair (ROME loop)

| Module | Iteration 0 | Iteration 1 |
|---|---|---|
| mux2_1, mux8_1, mux32_1 | Testbench ran successfully | - |
| mux4_1, mux16_1, mux64_1 | Error compiling testbench | Testbench ran successfully |
| half_adder, full_adder, adder4, adder8 | Testbench ran successfully | - |

Example repair: for mux4_1, iteration 0 declared `output reg out` and then drove it with a continuous assign. iverilog said `reg out; cannot be driven by primitives or continuous assignment`. ROME sent that error and the module back to the LLM. Iteration 1 compiled and passed. The TB was not changed during any of this.

PPA proxy results (best candidate per mux, Yosys `stat -tech cmos`):

| Module | mux2_1 | mux4_1 | mux8_1 | mux16_1 | mux32_1 | mux64_1 |
|---|---|---|---|---|---|---|
| Transistor estimate | 12 | 36 | 84 | 180 | 372 | 756 |
| Cells | 1 | 3 | 7 | 15 | 31 | 63 |
| Best iteration | 0 | 1 | 0 | 0 | 0 | 0 |

No timing proxy was reported because these modules are combinational.

## 5. How I integrated coverage_exp into ROME

I kept the same steps as coverage_exp: compile with iverilog -g2001, run with vvp to make a VCD, clean the VCD, run covered score, then covered report -m ltc in summary and detail mode. To get the waveform, I wrap the TB: the notebook makes a copy of the TB with one extra `initial $dumpfile/$dumpvars` line. The TB file used by ROME is not changed. The VCD cleanup is the same one the coverage_exp Makefile does: drop the Icarus `$comment` line and put `$timescale` on one line, because Covered 0.7.10 can't parse them. All four RTL modules go to Covered, so the whole tree under `tb_adder8.dut` is scored. I use the instance view (-i). The module view merges all 8 half adders into one row, which hides gaps. Line, toggle 0->1, toggle 1->0 and logic coverage are read from the Accumulated rows of the summary report.

For feedback, I keep only the miss lines from the detail report: the instance, the signals with their toggle masks, and the missed logic combinations. That goes to the LLM with the current TB. The prompt says to keep every existing line and only add new run_test calls. The notebook checks that every old run_test call is still there and that the new TB compiles. If not, the error goes back to the LLM. The RTL is frozen in this loop. If a stronger TB had failed, the TB would be frozen and the ROME verilog_loop would repair adder8 (I added an optional start-from-current-RTL argument to verilog_loop for that).

## 6. Iteration table (RTL changes vs TB changes)

| Step | What changed | RTL changed | TB changed | Result |
|---|---|---|---|---|
| ROME, all 4 adder modules | RTL generated | yes | no | PASS on iteration 0 |
| Coverage baseline | nothing (measure only) | no | no | PASS, 8 tests |
| Refine 1 (tb_iter_1.v) | LLM added 11 tests | no | yes | PASS, 19 tests |
| Refine 2 (tb_iter_2.v) | LLM added 2 tests | no | yes | PASS, 21 tests |

No step changes both RTL and TB. The stronger TBs passed on the existing RTL, so no RTL repair was needed in the coverage loop.

## 7. Coverage history (adder8, Covered instance view)

| Iteration | # tests | Line | Toggle 0->1 | Toggle 1->0 | Logic |
|---|---|---|---|---|---|
| Baseline | 8 | 100% | 79% | 69% | 72% |
| Refine 1 | 19 | 100% | 100% | 100% | 98% |
| Refine 2 | 21 | 100% | 100% | 100% | 99% |

**What drove Refine 1.** At baseline, a[7] and b[7] never went 0->1, bits 6 and 7 never went 1->0, and cout never toggled. So there was never a carry out of bit 7. Inside the low adder4, the first half adder's carry (carry1) in fa1-fa3 never toggled. The LLM added tests like FF+01, FF+FF+1, 80+80, F0+F0, C0+40 and 0F+0F. These drive the top bits, make carries ripple through both nibbles, and produce cout = 1.

**What drove Refine 2.** After Refine 1, only three logic combinations were missed, all in the high adder4. The sum XOR in fa2.ha1 and fa3.ha1 never saw (a,b) = (0,1), and the one in fa3.ha2 never saw (1,0). The LLM added 2 tests (C0+C0 and 80+00). 80+00 covered the fa3.ha2 case, so logic went from 98% to 99%. The LLM's comments targeted (1,1) for fa2.ha1 and fa3.ha1, but the actual misses were (0,1), so those two stayed uncovered.

Line coverage is 100% from the start because the RTL is all continuous assigns, which always run. Still missed at the end: (a,b) = (0,1) for the sum XOR in u_adder4_high.fa2.ha1 and fa3.ha1, meaning no test had a[6] = 0 with b[6] = 1, or a[7] = 0 with b[7] = 1.

## 8. Final commands and result

```
# coverage run (per iteration)
iverilog -g2001 -o coverage/<iter>/sim.vvp coverage/<iter>/rtl.v coverage/<iter>/tb_with_dump.v
vvp coverage/<iter>/sim.vvp
covered score -t adder8 -i tb_adder8.dut -v coverage/<iter>/rtl.v \
    -vcd coverage/<iter>/adder8.vcd -o coverage/<iter>/adder8.cdd
covered report -m ltc -d s -i -o coverage/<iter>/coverage_summary.rpt coverage/<iter>/adder8.cdd
covered report -m ltc -d v -i -o coverage/<iter>/coverage_detail.rpt coverage/<iter>/adder8.cdd

# final check
iverilog -g2001 -o coverage/final.vvp half_adder/half_adder.v full_adder/full_adder.v \
    adder4/adder4.v adder8/adder8.v testbenches/tb_final.v
vvp coverage/final.vvp
```

Final result: `tb_final.v` (21 tests) passes on the final RTL with `All tests passed!` Final coverage: line 100%, toggle 0->1 100%, toggle 1->0 100%, logic 99%.